An Experimental Comparative Analysis of TCP Reno, CUBIC, and BBR Under Varying Network Conditions

Aditya Singh Rana, Vinay Mor, Tejas Pisal

[Confirm department, institution, and author contact details before submission]

AUTHOR ACTION REQUIRED — This is a pre-submission research draft. No experimental measurements have been run in the current authoring environment. Run the supplied experiment package, replace every bracketed results item, update the conclusion, remove this note, and apply the target conference template before submission. Do not present placeholder values as results.

Abstract

TCP congestion-control algorithms can produce different throughput, delay, retransmission, and bandwidth-sharing behavior under the same network conditions. This paper specifies a reproducible experimental comparison of Linux TCP Reno, CUBIC, and BBR using a Mininet dumbbell topology with a shared bottleneck. The supplied experiment artifact automates one-factor-at-a-time variations in bottleneck bandwidth, configured link delay, random packet loss, and queue limit, and includes a mixed-flow experiment in which Reno, CUBIC, and BBR compete concurrently. Receiver-side throughput is collected using iperf3; sender retransmissions are extracted from iperf3 summaries; and TCP RTT and congestion-window samples are collected from Linux socket statistics where available. Raw JSON, per-run CSV data, environment metadata, confidence-interval summaries, and plots are retained to support auditability. Because the experiments have not yet been executed in the target Linux environment, this draft does not claim numerical performance findings. The final abstract must be updated from measured results and must state the principal quantitative findings rather than anticipated outcomes.

Index Terms—TCP congestion control, Reno, CUBIC, BBR, Mininet, throughput, round-trip time, fairness, reproducibility.

I. Introduction

Transmission Control Protocol (TCP) provides reliable, ordered delivery of byte streams, while congestion control limits the rate at which a sender injects data into the network [1], [2]. The congestion-control algorithm influences how quickly a connection uses available capacity, responds to congestion signals, recovers after loss, and shares a bottleneck with other flows. These effects depend on more than the algorithm label: kernel implementation, path delay, buffer configuration, traffic mix, and competing connections can change the observed outcome.

Reno represents the classical loss-based additive-increase/multiplicative-decrease family described by TCP congestion-control standards [2]. CUBIC changes congestion-window growth using a cubic function and is specified in RFC 9438 [3]. BBR instead estimates bottleneck bandwidth and minimum round-trip propagation time to pace traffic using a model of the path [4]. These design differences motivate a comparison that measures multiple metrics and avoids declaring one algorithm universally superior.

Prior evaluations have already examined BBR throughput, queuing delay, loss, and fairness [5], as well as the fairness of BBR and CUBIC flows sharing a bottleneck [6]. BBRv3 has also been evaluated under variations in buffer size, flow count, delay, and loss [7]. More recently, cloud-based work compared Reno, CUBIC, and BBR across synthetic throughput tests and application workloads [8]. Therefore, this study does not claim that a three-algorithm comparison is itself novel. Its intended contribution is a compact, reproducible laboratory artifact that applies a documented one-factor-at-a-time matrix, includes a mixed Reno/CUBIC/BBR sharing case, preserves raw observations, and generates summaries from the same data used in the manuscript.

The research questions are: (RQ1) How do receiver-side throughput, sampled TCP RTT, retransmissions, and congestion-window observations change as bandwidth, configured delay, loss, and queue limits vary? (RQ2) How is throughput divided when one Reno, one CUBIC, and one BBR flow concurrently share the same bottleneck? (RQ3) Can the resulting measurements, run settings, and analysis outputs be generated from a documented and repeatable procedure?

II. Background and Related Work

A. Congestion-control mechanisms

Reno uses loss indications to trigger window reduction and then increases its congestion window conservatively during congestion avoidance [2]. CUBIC defines a cubic window-growth function based on elapsed time since the last congestion event; the modern standards-track specification is RFC 9438, which updates RFC 5681 in its treatment of CUBIC behavior [3]. BBR estimates bottleneck delivery rate and minimum RTT and uses those estimates to control pacing rather than relying primarily on packet loss as its congestion signal [4]. The Linux system under test determines the concrete implementation of each algorithm; results must therefore report the kernel release and the congestion-control names exposed by the kernel.

B. Prior experimental evaluations

Hock, Bless, and Zitterbart independently evaluated BBR with high-rate bottlenecks, varying RTT, flow count, and buffer size. Their study reported the intended utilization behavior but also observed cases involving increased queuing delay, unfairness, and packet loss [5]. Sasaki et al. investigated concurrent BBR and CUBIC connections and reported substantial fairness problems in some high-latency conditions [6]. These findings motivate measuring both per-flow outcomes and aggregate performance, rather than throughput alone.

Gomez et al. evaluated BBRv3 alongside CUBIC under different flow counts, propagation delays, loss rates, buffer sizes, and queue-management schemes [7]. Swamy et al. subsequently compared Reno, CUBIC, and BBR in an AWS dumbbell environment across synthetic throughput and application workloads, illustrating that congestion-control behavior can differ across workload types [8]. Mininet enables small-scale network emulation using lightweight OS-level virtualization [9], but an emulated result remains conditional on the kernel, host scheduling, virtualization, and configured topology. The present work is best understood as a reproducibility-oriented controlled study, not as evidence that existing large-scale or workload-diverse studies are absent.

III. Study Design and Contributions

The study compares the Linux implementations identified by the kernel as `reno`, `cubic`, and `bbr`. It examines receiver-side throughput as the primary performance measure and uses sender retransmission counts, sampled TCP RTT, congestion-window samples, and mixed-flow fairness as complementary measures. The artifact contributes (i) an executable Mininet testbed with a single shared bottleneck, (ii) a documented scenario matrix that varies one primary impairment at a time, (iii) persistent raw and processed outputs, and (iv) an analysis script that reports mean, standard deviation, sample count, and 95% confidence intervals for repeated observations.

The contribution is methodological and empirical. A claim of algorithmic novelty is not made. Whether the resulting study is sufficiently novel for a particular conference must be assessed

against that venue's scope and the latest literature, especially because Reno/CUBIC/BBR comparisons and fairness studies already exist [5]–[8].

IV. Experimental Methodology

A. Testbed and topology

The provided implementation creates six Mininet hosts and two Open vSwitch switches. Three senders are attached to the left switch, three receivers are attached to the right switch, and all sender-to-receiver traffic crosses one configurable inter-switch bottleneck. Access links are configured above the bottleneck rate. The topology supports both isolated single-flow trials and three simultaneous competing flows. The design follows the common dumbbell arrangement used to make a shared contention point observable [5], [6], [8], [9].

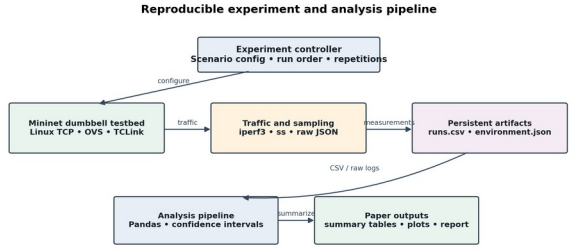


Fig. 1. Experiment automation, measurement, data retention, and analysis pipeline.

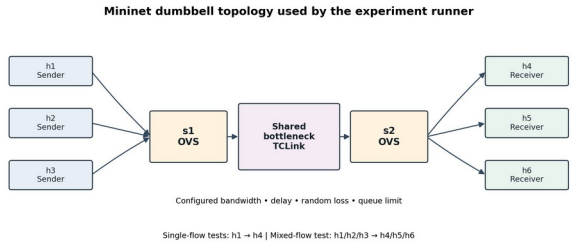


Fig. 2. Mininet dumbbell topology. The bottleneck is shared by all flows.

The script writes the kernel release, tool version, selected congestion-control algorithm names, scenario parameters, and experiment timestamps to the output directory. Before collecting publishable measurements, the operator must inspect the actual qdisc settings (for example, with `tc -s qdisc show` inside the relevant namespaces) and retain the environment file with the raw logs. Delay is controlled using Mininet's TCLink delay parameter; the configured parameter must not be reported as an observed end-to-end RTT. RTT is measured separately from sampled socket statistics.

B. Experimental parameters

A one-factor-at-a-time matrix varies bottleneck bandwidth, link delay, injected random loss, and queue limit while holding the other listed settings at their baseline values. The numerical settings below are the proposed default values in the supplied runner; they are experimental design choices, not measured findings. A full execution uses 30-second transfers, omits the initial 3 seconds from iperf3's measurement interval, and repeats each algorithm/scenario combination five times. The script randomizes algorithm order within each scenario/repetition using a recorded seed.

Scenario	Varied setting	Settings held at baseline
Baseline	None	20 Mbit/s; 10 ms; 0% loss; 100-packet queue
Low bandwidth	5 Mbit/s	10 ms; 0% loss; 100-packet queue
High delay	50 ms	20 Mbit/s; 0% loss; 100-packet queue
Random loss	0.5% loss	20 Mbit/s; 10 ms; 100-packet queue
Small queue	20 packets	20 Mbit/s; 10 ms; 0% loss

TABLE I. Default one-factor-at-a-time experimental matrix configured by the supplied artifact.

For single-flow experiments, h1 sends a bulk TCP transfer to h4. The sender is configured with one of the three congestion-control algorithms before each trial. For mixed-flow fairness, h1, h2, and h3 simultaneously transfer to h4, h5, and h6, respectively, using Reno, CUBIC, and BBR. A mixed-flow repetition shares a common run identifier so that throughputs are grouped correctly for fairness analysis. The test runner records one output row per flow; raw iperf3 JSON is retained independently.

C. Metrics and measurement procedure

Receiver-side throughput is calculated from iperf3's `end.sum_received.bits_per_second` field, converted to Mbit/s. Sender throughput is retained as a cross-check. Sender retransmissions are extracted from the iperf3 sender summary when exposed by the installed version. These counts are not equivalent to the configured random-loss percentage: retransmissions depend on transport behavior and measurement details.

TCP RTT and congestion-window values are sampled from `ss -tin` while a flow is active. The runner stores the mean, median, and 95th percentile of captured RTT samples, and the median and maximum captured congestion-window values. The sampling interval is approximately 350 ms; fields may be absent on systems whose socket reporting differs or if the connection is too short to sample. Missing values remain empty rather than being replaced with zero. The standard iperf3 jitter field is not used as a TCP jitter measurement; a separate UDP workload would be needed if jitter were a research objective.

For a mixed-flow run with throughputs x1, ..., xn, Jain's fairness index is computed as J = (sum(xi))^2 / [n sum(xi^2)] [10]. The index lies between 1/n and 1 for positive allocations, with 1 indicating equal throughput across the measured flows. Fairness is interpreted alongside total throughput because equal shares do not, by themselves, imply high link utilization.

D. Repetition and analysis

The experiment runner uses repeated independent runs for each condition. The analysis program aggregates each scenario/algorithm group and reports the number of valid observations, mean, sample standard deviation, and a two-sided 95% t-based confidence interval for metrics with sufficient non-missing values. For mixed-flow tests, the analysis computes one fairness index per simultaneous-run identifier. The generated figures include mean values with confidence intervals where calculable. Any exclusion of failed or incomplete runs must be documented; the default analysis does not silently replace missing measurements.

Run ordering, host workload, kernel version, queue discipline, CPU scheduling, segmentation/offload settings, and virtualization configuration can influence the result. The code records the kernel and tool versions but does not fully normalize every hardware or offload property. Before the final study, the authors should record `ethtool -k` output where relevant, inspect qdiscs, keep the machine otherwise idle, and report any configuration changes. The analysis is descriptive and does not automatically establish statistical significance between algorithms.

V. Results and Discussion

RESULTS REQUIRED — This section must be completed after running `src/run_experiments.py` on the documented Linux/Mininet environment. Use `src/analyze_results.py` to create the measured summaries and plots. Do not submit this manuscript while the bracketed items and this instruction remain.

A. Single-flow performance

Report receiver-side throughput by algorithm and scenario using the generated `summary_by_scenario_algorithm.csv` and the corresponding throughput plot. Include observed sample counts and 95% confidence intervals. Then explain the measured changes

across the bandwidth, delay, loss, and queue settings without generalizing beyond the test conditions.

[INSERT: measured throughput table and Figure 3 generated from the full experimental runs. Summarize actual differences with units, n, and uncertainty.]

B. Delay, retransmissions, and congestion window

Use the RTT, retransmission, and congestion-window plots only where the raw logs contain valid measurements. Compare trends with the algorithm designs and prior evaluations [4]–[7], while noting that `ss` sampling is periodic rather than packet-by-packet tracing. Explain any absent or anomalous measurements and verify them against the corresponding raw JSON and system logs before drawing conclusions.

[INSERT: measured RTT/retransmission/cwnd findings and figures. State how many observations were available for each metric; do not treat configured loss as measured packet loss.]

C. Mixed-flow fairness

For the mixed-flow experiment, report each flow's measured throughput, aggregate throughput, and Jain's fairness index across repeated concurrent runs. Explain which flow received what share under this particular bottleneck, and compare the result cautiously with prior BBR/CUBIC fairness studies [5]–[7].

[INSERT: measured mixed-flow fairness table and `jain_fairness.png`; report n and confidence interval.]

D. Interpretation

The final discussion should answer RQ1–RQ3 directly. Distinguish observations from explanations: measured throughput or RTT is an observation, while attributing it to pacing, loss response, or window growth is an interpretation that should be supported by the recorded configuration and relevant literature. If outcomes vary across repetitions or conflict with expected behavior, report the variation rather than discarding inconvenient runs without a documented rule.

VI. Threats to Validity and Limitations

First, Mininet shares a host kernel and physical CPU resources among virtual nodes; its timing and throughput may be affected by host scheduling and virtualization overhead. Second, results depend on the specific Linux kernel's Reno, CUBIC, and BBR implementations. The label `bbr` alone does not establish that the tested algorithm is BBRv1, BBRv2, or BBRv3; the version should be verified from kernel documentation or build metadata where possible. Third, the matrix uses a limited set of one-factor-at-a-time configurations and does not cover every interaction between bandwidth, delay, loss, buffer size, active queue management, and flow count. Fourth, the short set of bulk iperf3 transfers is not representative of all application workloads, wireless paths, Internet routes, or short-flow behavior. Finally, socket RTT/cwnd sampling can miss fast transient events. Accordingly, conclusions should be restricted to the configured emulated testbed and verified measured results.

VII. Conclusion

CONCLUSION REQUIRED — Replace this instruction after the experiment run. State the key measured finding for each research question, quantify the most important differences, acknowledge uncertainty and scope, and identify one or two evidence-based future-work directions. Do not state that BBR, CUBIC, or Reno is universally superior.

Artifact and Data Availability

The experiment source code, configuration matrix, analysis script, and instructions are supplied with this manuscript draft. For a public conference submission, archive the code and final raw/processed dataset in a stable repository and replace this sentence with the permanent URL and version/commit identifier. Do not publish sensitive host data; the intended artifacts here are network metrics, tool/kernel metadata, and experiment outputs.

References

[1] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, Aug. 2022, doi: 10.17487/RFC9293.

[2] M. Allman, V. Paxson, and E. Blanton, “TCP Congestion Control,” RFC 5681, Sep. 2009, updated by RFC 9438.

[3] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, “CUBIC for Fast and Long-Distance Networks,” RFC 9438, Aug. 2023.

[4] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, “BBR: Congestion-Based Congestion Control,” Communications of the ACM, vol. 60, no. 2, pp. 58–66, 2017, doi: 10.1145/3009824.

[5] M. Hock, R. Bless, and M. Zitterbart, “Experimental evaluation of BBR congestion control,” in Proc. IEEE 25th International Conference on Network Protocols (ICNP), 2017, pp. 1–10, doi: 10.1109/ICNP.2017.8117540.

[6] K. Sasaki, M. Hanai, K. Miyazawa, A. Kobayashi, N. Oda, and S. Yamaguchi, “TCP Fairness Among Modern TCP Congestion Control Algorithms Including TCP BBR,” in Proc. IEEE 7th International Conference on Cloud Networking (CloudNet), 2018, pp. 1–4, doi: 10.1109/CloudNet.2018.8549505.

[7] J. Gomez, E. F. Kfoury, J. Crichigno, and G. Srivastava, “Evaluating TCP BBRv3 performance in wired broadband networks,” Computer Communications, vol. 222, pp. 198–208, Jun. 2024, doi: 10.1016/j.comcom.2024.04.037.

[8] Aravinda Swamy D. et al., “Performance evaluation of TCP congestion control variants across application workloads in cloud based networks,” Scientific Reports, vol. 16, Art. no. 31639, Jul. 2026, doi: 10.1038/s41598-026-61100-w.

[9] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in Proc. 10th ACM Workshop on Hot Topics in Networks (HotNets), 2010, doi: 10.1145/1868447.1868466.

[10] R. Jain, D. Chiu, and W. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems,” DEC Research Report TR-301, Sep. 1984.